

Sabancı University

Faculty of Engineering and Natural Sciences

Computer Science and Engineering

PixelProof AI Image Detection and Reliability Evaluation

CS395 Internship Final Report

Efe Han Keleş

Student ID: 31994

Company: Türk Telekomünikasyon A.Ş.

Supervisor: Önder Çelebi

Supervisor title: Director

Internship: 20 July–18 September 2026

Approved duration: 40 internship days

Format: Hybrid

Submission date: 29 September 2026

Abstract

Türk Telekom provides telecommunications and digital services across Türkiye. During my CS395 internship, I developed PixelProof to investigate AI-generated image detection while reducing false alerts on real photographs. A second experimental module explored locating AI-edited regions. Development followed an iterative process: inspect errors, define the next experiment, measure the result and record the decision. Dataset records tracked acquisition and permitted use, while experiment and history logs retained both improvements and failed approaches. Early transfer failures and label errors shifted the focus toward data quality and reliable evaluation. E92's learned components used 12,269 training images represented under four processing conditions, producing 49,076 training views. A separate but repeatedly used development collection contained 160 real and 160 AI images. E92 detected 159 AI images in each of two evaluation conditions. Compared with E43, false alerts fell from 68 to zero on real originals and from 68 to 14 after resizing and JPEG compression. All 20 numerical checks passed, but one newly missed reference-detected AI image prevented full acceptance. The deliverables include trained research components, a local web demonstration and reproducible records. Model2 remains experimental. The project demonstrates improvement on a defined development collection, rather than universal detection or production readiness. Further work should use independent, diverse evaluation sources, keep calibration separate from evaluation and test localization with more editors.

Table of Contents

1. Introduction.....	4
2. Company information.....	4
2.1 Corporate profile and services.....	4
2.2 Organization and internship context.....	5
3. Project background.....	5
3.1 Department information.....	5
3.2 Initial project status.....	6
3.3 Motivation and problem definition.....	6
3.4 Related literature.....	6
4. Internship project.....	7
4.1 Project objective and scope.....	7
4.2 My responsibilities.....	7
4.3 Methodology and tools.....	8
4.4 Expected outcomes and deliverables.....	9
4.5 Project details.....	9
4.5.1 Early models and the first transfer failures.....	9
4.5.2 Data integrity and corrected conclusions.....	9
4.5.3 Data roles and processing conditions.....	10
4.5.4 E92 representation and learning.....	11
4.5.5 Development improvement and acceptance limits.....	12
4.5.6 Honest scores and the web demonstration.....	13
4.5.7 Broader checks and rejected improvements.....	14
4.5.8 Model2 localization and its limits.....	14
4.6 Results.....	16
5. Internship experience.....	16
5.1 Learning.....	16
5.2 Relation to undergraduate education.....	17
5.3 Major difficulties.....	17
5.4 A typical day.....	17
6. Conclusions.....	18
7. Recommendations.....	18
8. References.....	19
9. Appendices.....	21
9.1 Development criteria.....	21
9.2 Evidence and reproduction map.....	21

1. Introduction

PixelProof investigates whether a photograph contains evidence of AI generation while limiting false accusations against real images. It also explores locating AI-edited regions. The main result is a working research demo with improved detection on a defined development collection; reliable performance across all image sources remains unproven.

My CS395 internship at Türk Telekom ran from 20 July to 18 September 2026 under an approved 40-day arrangement. This report covers completed research through 16 September. Experiment identifiers refer to protocols in the project records, not independent tests (Keleş, 2026). Sections 2 and 3 introduce the company and problem; Section 4 explains development and results; Sections 5–7 cover experience, conclusions and recommendations. References and an appendix support the account.

2. Company information

2.1 Corporate profile and services

Türk Telekomünikasyon A.Ş. provides fixed, mobile, broadband, television and corporate data services to households, businesses and wholesale operators. Its history exceeds 185 years; its services were brought under one brand in 2016. It operates offices and communications infrastructure across 81 provinces, with 31,756 employees reported alongside June 2026 figures (Türk Telekom, 2026b).

Its registered address is Turgut Özal Bulvarı, 06103 Aydınlikevler, Ankara. My internship office was Fatih Sultan Mehmet Mah., Balkan Cad. No:49, 34771 Ümraniye, İstanbul. The website is <https://www.turktelekom.com.tr>; the supplied internship contact number is +90 312 555 93 92. Ownership is 60% Türkiye Wealth Fund, 25% Treasury and Finance Ministry and 15% publicly traded shares. Subsidiaries include TT Mobil, TTNET, Argela, İnnova, SEBİT and AssisTT (Türk Telekom, 2026a).

Turkcell was a relevant competitor discussed during the internship. Nokia supplies fixed and mobile network equipment and related services to the Türk Telekom group (Finnvera, 2025).

2.2 Organization and internship context

Figure 1 summarizes the published corporate structure: network and IT support infrastructure, commercial units serve customers, and support units cover finance, staff and coordination (Türk Telekom, 2026a, p. 33). My host unit was Planlama ve Geliştirme Müdürlüğü (Planning and Development Directorate), led by Önder Çelebi.

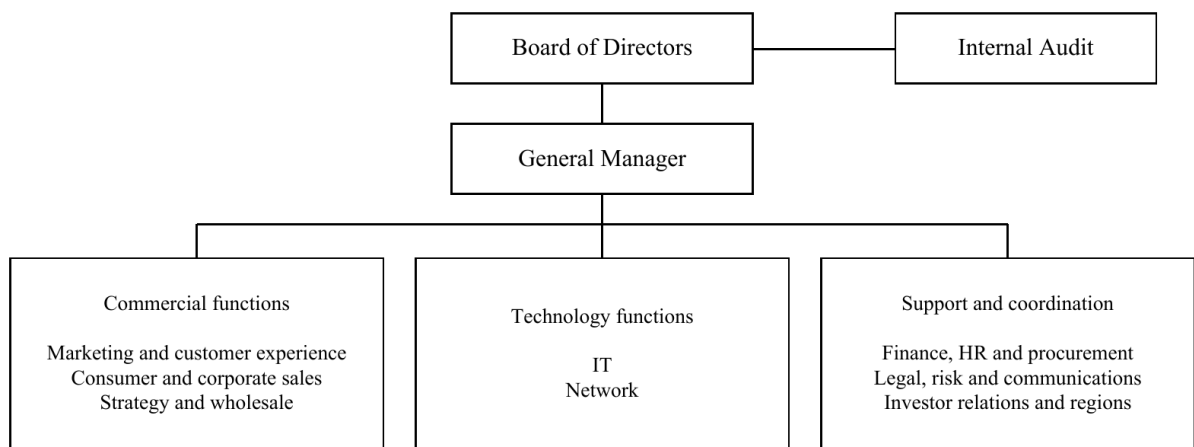


Figure 1. Simplified corporate reporting structure from the 2025 Annual Report.

3. Project background

3.1 Department information

My host unit, the Planning and Development Directorate, worked on Tivibu software and hardware. My supervisor was Önder Çelebi, Director, at onder.celebi@turktelekom.com.tr. Hasan Çontuk (hasan.contuk@turktelekom.com.tr), a member of Önder Çelebi's team, was my mentor. I consulted him for guidance while making the project decisions myself. I worked individually under a hybrid arrangement, Monday–Thursday on site and Friday remote. PixelProof was a student prototype using research data, with no documented company production deployment.

3.2 Initial project status

The initial workflow was to prepare labelled images, train a small CNN on CIFAKE and inspect classification errors. Its 96.75% familiar-test accuracy fell to 77.1% on a separate 995-image collection. Later audits found confounds in that collection, so these remain historical diagnostics. Data flowed through local preparation and training scripts to saved scores, error analysis and the next experiment (Keleş, 2026, E1 and E10).

3.3 Motivation and problem definition

A false positive flags a real photograph as AI; a false negative misses an AI image. Improving one error can worsen the other. The main difficulty was distribution shift: a new camera, generator or processing method could change performance. Model1 addressed whole-image detection, while Model2 investigated edited regions using pixel masks. A low detection score cannot prove authenticity.

In this report, generalization means working on image sources that were not used to guide development. A detector may learn a camera or compression pattern instead of a reliable sign of generation. If that pattern changes, its predictions can change even when the image still looks similar to a person. This is why the project compared sources and processing conditions separately.

3.4 Related literature

Wang et al. (2020) investigated detection across different image generators. Ojha et al. (2023) showed that pretrained visual features can support broader detection through simple classifiers. CLIP learns from image–text pairs (Radford et al., 2021), while DINOv2 learns reusable features without task-specific labels (Oquab et al., 2024). These works motivated feature-based transfer learning in PixelProof.

Compression and image-size differences can become misleading dataset shortcuts (Grommelt et al., 2024). DEAR prunes features using inpainting-based analysis to improve robustness

(Kim et al., 2026). PixelProof used a pretrained DEAR component, but its published benchmark results do not establish our system's performance.

SIDD provides smartphone images from ten scenes and five cameras (Abdelhamed et al., 2018); SID provides short- and long-exposure low-light raw pairs (Chen et al., 2018). These sources supported real-image coverage, with limited scene diversity and processing conditions that differ from ordinary camera JPEGs.

Guo et al. (2017) explain why classifier scores need calibration before interpretation as confidence. Dwork et al. (2015) show the risks of repeatedly adapting to holdout results. Both issues shaped our evaluation and interface. As an industry comparison, Adobe Content Credentials records origin and editing information (Adobe, 2026). PixelProof estimates evidence from pixels; credential verification was not implemented.

The literature suggested useful starting points, but it could not settle the project question. A method that performs well in its own paper may use different generators, cameras or decision rules. I therefore used these studies to choose experiments and explain possible failures. The report separates published findings from measurements made in PixelProof, so that the reputation of a pretrained model does not stand in for local evidence.

4. Internship project

4.1 Project objective and scope

Model1 aimed to reduce real-photo false alerts while preserving AI detections already made by a reference model. Model2 explored edit localization. The scope covered data auditing, learned adaptation, evaluation and a local web demo with uncertainty handling. Universal detection and production readiness were longer-term goals, not achieved outcomes.

4.2 My responsibilities

I directed the experiments, organized data, reviewed results and generally made project decisions, consulting my supervisor and mentors for advice. They explained concepts such as

true positives and false negatives. My work covered dataset checks, model adaptation, the API, interface and research records. AI coding assistance supported implementation, analysis and documentation; scripts and tests checked outputs. Pretrained models and external datasets are credited separately from our adaptation and application work.

4.3 Methodology and tools

In later experiments, PLAN.md recorded the question and acceptance rules before a run. ml/EXPERIMENTS.md retained measurements, HISTORY.md explained decisions and DATASETS.md recorded sources and permitted use. Corrections remained alongside earlier entries. Table 1 shows how findings changed the next step (Keleş, 2026).

Table 1. Examples of findings that changed development.

Finding	Response
Reversed source labels	Remap labels and rerun experiments
Evaluation-dependent threshold	Use calibration data only
Better average but new AI misses	Reject the candidate

Training fitted the model; calibration selected thresholds; development comparisons guided changes. Final evaluation required data uninvolved in those decisions. I tracked AI recall, real-image false alerts and AUC, which measures ranking across thresholds. Python, PyTorch, NumPy, SciPy and scikit-learn supported modelling; React, TypeScript and FastAPI supported the demo. Git, saved scores, artifact hashes and automated checks made changes traceable.

For each candidate, the practical question was whether the change solved a known problem without creating an unacceptable one elsewhere. A lower total error count was insufficient if the model newly missed an AI image that the reference had detected. Keeping the failed candidates in the records helped explain why a newer experiment did not automatically replace the current demo model.

4.4 Expected outcomes and deliverables

The deliverables were trained research components, a local upload demo, reproducible experiment scripts and documented data and results. Model2 contributed an experimental localization evaluator. The prototype was intended for demonstration and further study.

4.5 Project details

4.5.1 Early models and the first transfer failures

The first CNN used 90,000 CIFAKE training images, 10,000 validation images and a 20,000-image familiar test. A pretrained ResNet-18 reached 97.66% on the familiar test but only 25.2% on the separate 995-image collection. Matching the training resolution recovered much of this loss. A stronger architecture alone had not solved source transfer (Keleş, 2026, E1–E6).

Native-resolution statistics, tile features, external detectors and combined scores were then compared. Gains on particular sources often came with new real-photo errors or missed AI types. Fixed crops and camera-noise explanations also needed controls rather than being assumed reliable. These failures shifted attention toward data quality, source coverage and explicit acceptance rules (Keleş, 2026, E8–E50).

I treated the familiar test as a check that the training procedure worked on similar data. The external collection asked a harder question about transfer to other sources. The large gap between them showed why one accuracy number was not enough. It also motivated checking input preparation before making a more expensive model change.

4.5.2 Data integrity and corrected conclusions

Two datasets used numeric labels opposite to our convention of 0 for real and 1 for AI. Explicit mapping and reruns corrected affected experiments. DINOv2’s Defactify AUC changed from 0.480 to 0.764, invalidating an earlier explanation that its features were unsuitable. The larger training pool also reduced false alerts more than first reported. A

separate threshold-selection procedure was repaired because it could consult evaluation data; the corrected candidate failed admission and was removed (Keleş, 2026, E19b–E19c and E27).

A label is the answer supplied during learning or evaluation. Reversing its meaning can make a useful model appear poor and can teach a new model the wrong relationship. The repair therefore required more than changing a chart caption. Affected measurements had to be rerun, and the earlier explanation had to be corrected in the written record.

4.5.3 Data roles and processing conditions

A parent is one image before project-created transformations. E92 training used 12,269 parents: 7,674 real and 4,595 AI. Four processing conditions produced 49,076 views. These comprised clean input, an assigned transport transformation, JPEG75 and a social-style view capped at 1080 pixels before JPEG75 encoding. Downloaded archive size, selected training data and evaluation size are different quantities. Source, licence, duplicate and known ancestry checks governed admission.

Table 2 separates the major populations; its rows are not additive independent evidence. Earlier E42 and E43 evaluations failed acceptance. E43 passed 11/20 checks on 1,000 real plus 1,000 AI images, with 391 original and 490 processed real false alerts. E92 never reached that final stage because its preceding acceptance rule failed (Keleş, 2026, E42, E49, E92 and E139).

Table 2. Training and evaluation populations at different stages.

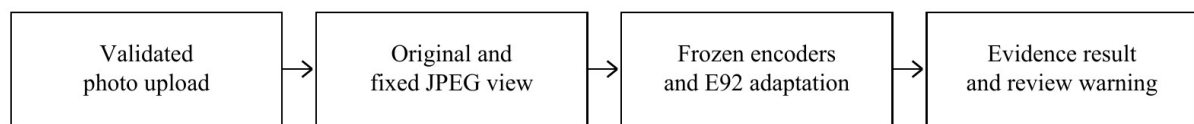
Population	Parents	Views	Role
E42 evaluation	16,953	50,858	Earlier model evaluation
E43 on E49-C	2,000	4,000	Earlier final evaluation
E92 training	12,269	49,076	Training
E92 on E66	320	640	Reused development
Later research	12,525	50,100	Internal diagnostics

The same photograph can appear at several sizes or compression levels. These versions help reveal sensitivity to processing, but they still come from one parent. Similarly, different photographs of the same scene may share useful clues. Counting views, files or downloaded gigabytes as independent examples would overstate the strength of the evidence. The useful question is which sources, scenes and generators a measurement actually covers.

4.5.4 E92 representation and learning

Fine-tuning, grayscale inputs and early score corrections failed the full requirements. Later work added CLIP and DEAR features and learned adaptation; E83 and E86 passed 17/20 development checks before E92 reached 20/20. This sequence alone does not isolate which addition caused improvement (Keleş, 2026, E51–E92).

E92 uses feature-based transfer learning. Pretrained DINOv2, CLIP and DEAR components remain fixed, while project-trained components learn to use their features. A constrained correction adjusts the reference score to reduce real-image errors while protecting AI responses on training examples. That protection still needs evaluation on other images. Figure 2 shows how the learned system connects to the interface.



Raw model scores remain separate from authenticity claims.

Figure 2. PixelProof inference flow from upload to displayed evidence.

Transfer learning reuses knowledge learned before this project. Here, an encoder converts pixels into a numerical description of the image. Keeping the encoder fixed means its pretrained weights stay unchanged while smaller project components learn from those descriptions. This made adaptation practical on the available computer. It also means that our contribution is the adaptation, evaluation and application around the existing representations, rather than the creation of DINOv2, CLIP or DEAR.

4.5.5 Development improvement and acceptance limits

Figure 3 compares the same 160 real development images. False alerts fell from 68 to zero for originals and from 68 to 14 for processed views. E92 detected 159 of 160 AI images in each condition. Ten numerical requirements per condition passed, but one original AI image previously detected by E43 became a new miss. The separate retention rule therefore failed.

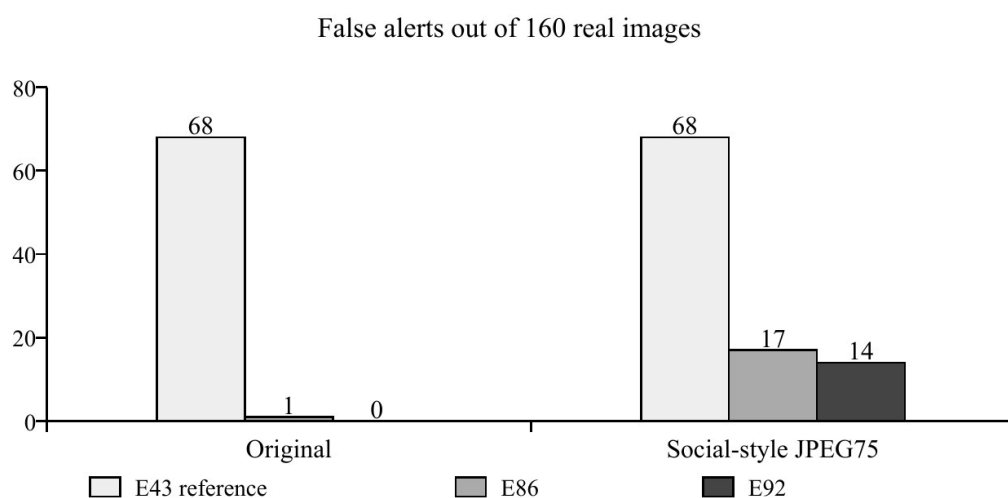


Figure 3. False alerts among 160 real development images per condition.

This reused collection contains only ten real SIDD scenes and two previously seen AI families. Thirteen of the fourteen processed real errors cluster in two scenes. Repeated use

influenced later decisions, limiting generalization claims. E102 lowered processed false alerts to 12/160 but retained the same reference-relative AI miss, so it was not promoted (Keleş, 2026, E92 and E102).

The comparison asks both how many errors occurred and which images caused them. Two models can detect the same number of AI images while missing different examples. Under the retention rule, recovering one example cannot compensate for newly missing another. This explains how E92 could improve the overall measurements and still fail the full decision rule. It also keeps the report from treating 20/20 as proof of perfect classification.

4.5.6 Honest scores and the web demonstration

The demo's 0–100 scale is a model score, not an AI probability. Its upper threshold is about 7.94, inherited from E48 calibration; its lower threshold is about 1.15, from already-used E49 calibration/development analysis. An original score at or above the upper threshold retains an AI alert. Two E92 scores below the lower threshold yield no clear AI evidence; other cases are uncertain. E43 provides advice without vetoing two low E92 scores. Decisions use unrounded values.

On the same development collection, the combined rule gave 139 real images no clear AI evidence and left 21 uncertain, with zero real AI alerts. It alerted on 159 AI images and left one uncertain. Thus per-view false alerts and final website warnings are different measurements. Upload validation, artifact checks and cancellation handling improve software operation, not measured detection accuracy (Keleş, 2026, current-policy audit).

The second processed view acts as a consistency check. When ordinary resizing and compression change the response, the user needs to know that the evidence is unstable. The interface therefore has an uncertain outcome instead of forcing every upload into a real-or-AI answer. Even the no-clear-evidence result only describes what the detector found. Establishing where a photograph came from requires additional information about its source.

4.5.7 Broader checks and rejected improvements

The owner gallery produced 9 false alerts among 206 real originals and 18 after processing. Mostly one phone and related scenes, it had already influenced development and was not independent. Separate source-fold research models also failed full acceptance; their results do not measure active E92 performance (Keleş, 2026, E95 and E146).

A processing audit found known 224-pixel derivatives in 29.18% of real training images versus 7.68% of AI images. Added source-pixel features slightly improved clean averages but harmed processed AI detection; one JPEG75 result fell from 75.73% to 74.15%. Follow-up decomposition identified harmful score contributions, without proving a physical cause. The candidate was rejected. Unsourced reserves remained, but no balanced, independent final test was established (Keleş, 2026, E148–E152).

These checks changed the next development priority. Adding another feature was less useful than learning whether the apparent improvement survived a change in source or processing. A suitable final evaluation must therefore be planned before inspecting its results. Its images should not have influenced training, threshold selection or decisions about which candidate to keep. More files from the same familiar sources would not resolve this problem by themselves.

4.5.8 Model2 localization and its limits

Model2 predicts edited regions and checks authentic images for false alarms. An early overlap evaluation improperly used true mask size to choose predicted area. A corrected test on 120 CocoGlide images performed worse than a central-region baseline (Keleş, 2026, E17 and E107).

Finding data was not the same as finding suitable training evidence. Of 512 CocoGlide originals, 506 shared source identities with protected project data, preventing new training admission under our split policy. The 512 downloaded DiffSeg30k pairs included 201 partial,

224 full-image and 87 empty masks; original-image ancestry and mask interpretation remained unresolved. They were kept out of training. Thus the project lacked a broad, validated local-edit dataset, despite having image files and masks (Keleş, 2026, E105, E109 and E117).

A local editing request also did not guarantee an unchanged background. In our Stable Diffusion 1.5 pilot, the 11 accepted outputs from 16 attempts changed an average of 99.94% of pixels outside the intended mask. Mean absolute channel change was about 9.79 on a 0–255 scale. Pixel differences do not mean that the scene changed semantically, but the raw background could not be labelled untouched. Explicit composites restored original pixels outside the mask; their boundaries still needed controls (Keleş, 2026, E122 and E127).

Revised input preparation later passed all 16 generation checks. We used these controlled composites, authentic images and conventional-edit controls for a small learning pilot. The masks defined the pasted region, not a verified semantic-change boundary. One editor and sixteen reused parents could not establish broad reliability (Keleş, 2026, E129–E135).

Table 3 shows that training on two edit placements improved the new placement but harmed the original placement and increased false markings on authentic images. The candidate was rejected. Moving edits also changes content, so the result does not isolate location as the sole cause. Model2 remains experimental.

Table 3. Model2 trade-offs on sixteen reused parents, original-image condition.

Endpoint	Earlier head	Two-placement head
Original-placement pixel AUC	0.74	0.70
New-placement pixel AUC	0.57	0.64
Authentic falsely flagged area	17.11%	25.74%

A localization map gives a score at each location. To assess it, we need a mask identifying the edited pixels and authentic images that should not contain an edited region. Pixel AUC measures how well scores rank edited pixels above authentic ones across thresholds. It is not the percentage of an edit correctly outlined. False markings on authentic images provide a separate practical check.

4.6 Results

The completed outcome is an E92 research system, local demo and reproducible development record. Table 4 summarizes per-view results on 160 real plus 160 AI development images, not the paired website outcomes in Section 4.5.6.

Table 4. E92 per-view results on the reused development collection.

Measure	Original	Social-style
AI detection	159/160 (99.38%)	159/160 (99.38%)
Real false alerts	0/160 (0.00%)	14/160 (8.75%)
Balanced accuracy	99.69%	95.31%
Numerical criteria	10/10	10/10
New AI misses versus E43	1	0

The numerical improvement did not meet full acceptance or prove universal reliability. Model2 also remained experimental. The engineering checkpoint passed 1,245 Python tests, which check software behavior rather than image accuracy. No production rollout or company deployment impact was measured (Keleş, 2026).

5. Internship experience

5.1 Learning

I learned to ask what a good score actually proves. Early accuracy looked promising, but failures on new sources and the label correction changed my conclusions. I also learned to sustain an individual project, reduce scope when it became too complex and ask focused

questions. Practical experiments made machine learning concepts clearer and strengthened my existing interest in the field.

The records also helped me manage the work. After a run, I needed to explain what changed, what the result supported and what should happen next. Writing that decision made it easier to return to the project after an interruption and avoid repeating a rejected approach. I learned that narrowing a claim can be more useful than defending an attractive but weak result.

5.2 Relation to undergraduate education

CS412 Machine Learning provided the concepts for the first experiments. In PROJ201, our team trained YOLOv11 on a custom dataset and integrated it into an application intended to guide visually impaired users; we placed first in our group. DSA210 provided foundations in data science. The internship showed that I also needed stronger preparation in experimental design and data provenance, especially label consistency and shared image origins.

5.3 Major difficulties

The first difficulty was source transfer. Expanding documented coverage and inspecting group-level errors helped, but generalization remains unresolved. The second was measurement integrity: explicit label mapping, isolated calibration and reruns corrected misleading results. The third was local resource and runtime limits. Feature caching and bounded batches helped; a memory failure required a separately recorded implementation repair. Cancellation and artifact checks improved operation without changing the model.

5.4 A typical day

On office days, I usually arrived at 7:30 a.m. After breakfast, I worked through the morning until around noon or early afternoon, then continued with occasional tea or coffee breaks. Work involved reviewing the plan, implementation, experiments, interpretation and updating records. I asked my supervisor or mentors for clarification when needed and then decided how to proceed. Fridays were remote.

6. Conclusions

PixelProof became a working research prototype that substantially reduced real-image false alerts on a defined development collection. Its value also lies in the record of corrections and rejected ideas. The remaining reference-relative AI miss, repeated development-data use and later source failures prevent a claim of universal reliability. Model2 needs broader evidence before use beyond experiments.

7. Recommendations

Future work should establish an independent evaluation with diverse cameras and generators, verified source ancestry and calibration separated from evaluation. Model2 needs more independent parents and editors, including authentic and conventional-edit controls. Future interns should agree on scope early, learn to reproduce saved results, keep failures in the record and communicate what the evidence supports. AI assistance requires checking its output and understanding the work personally.

8. References

- Abdelhamed, A., Lin, S., & Brown, M. S. (2018). A high-quality denoising dataset for smartphone cameras. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 1692–1700. Retrieved September 29, 2026, from <https://abdokamel.github.io/sidd/>
- Adobe. (2026, August 25). Content Credentials overview. Adobe Help Center. Retrieved September 29, 2026, from <https://helpx.adobe.com/creative-cloud/apps/adobe-content-authenticity/content-credentials/overview.html>
- Chen, C., Chen, Q., Xu, J., & Koltun, V. (2018). Learning to see in the dark. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 3291–3300. Retrieved September 29, 2026, from <https://cchen156.github.io/SID.html>
- Dwork, C., Feldman, V., Hardt, M., Pitassi, T., Reingold, O., & Roth, A. (2015). Generalization in adaptive data analysis and holdout reuse. *Advances in Neural Information Processing Systems*, 28. Retrieved September 25, 2026, from <https://proceedings.neurips.cc/paper/2015/hash/bad5f33780c42f2588878a9d07405083-Abstract.html>
- Finnvera. (2025, October 23). Finnvera guarantees Nokia’s deliveries to Türk Telekom. Retrieved September 29, 2026, from <https://www.finnvera.fi/eng/export-and-internationalisation/finnvera-as-an-export-finance-provider/guaranteed-transactions/finnvera-guarantees-nokias-deliveries-to-turk-telekom>
- Grommelt, P., Weiss, L., Pfreundt, F.-J., & Keuper, J. (2024). Fake or JPEG? Revealing common biases in generated image detection datasets. *arXiv:2403.17608*. Retrieved September 25, 2026, from <https://arxiv.org/abs/2403.17608>
- Guo, C., Pleiss, G., Sun, Y., & Weinberger, K. Q. (2017). On calibration of modern neural networks. *Proceedings of Machine Learning Research*, 70, 1321–1330. Retrieved September 25, 2026, from <https://proceedings.mlr.press/v70/guo17a.html>
- Keleş, E. H. (2026). PixelProof experiment, history and dataset records [Software and research records, commit 497d45c]. GitHub. Retrieved September 25, 2026, from <https://github.com/EfeHanKeles346/ai-image-detector/tree/497d45c>
- Kim, D., Choi, J., Seong, H. S., Kim, S., Lee, D., Yi, S., & Choi, J.-H. (2026). Dissect and prune: Enhancing robustness in AI-generated image detection. *ICML 2026*; *arXiv:2606.10309*, version 2. Retrieved September 25, 2026, from <https://arxiv.org/abs/2606.10309v2>
- Ojha, U., Li, Y., & Lee, Y. J. (2023). Towards universal fake image detectors that generalize across generative models. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 24480–24489. Retrieved September 25, 2026, from <https://arxiv.org/abs/2302.10174>

- Oquab, M., Darcet, T., Moutakanni, T., Vo, H., Szafraniec, M., Khalidov, V., Fernandez, P., Haziza, D., Massa, F., El-Nouby, A., Assran, M., Ballas, N., Galuba, W., Howes, R., Huang, P.-Y., Li, S.-W., Misra, I., Rabbat, M., Sharma, V., Synnaeve, G., Xu, H., Jégou, H., Mairal, J., Labatut, P., Joulin, A., & Bojanowski, P. (2024). DINOv2: Learning robust visual features without supervision. *Transactions on Machine Learning Research*. Retrieved September 25, 2026, from <https://arxiv.org/abs/2304.07193>
- Radford, A., Kim, J. W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., Sastry, G., Askell, A., Mishkin, P., Clark, J., Krueger, G., & Sutskever, I. (2021). Learning transferable visual models from natural language supervision. *Proceedings of Machine Learning Research*, 139, 8748–8763. Retrieved September 25, 2026, from <https://proceedings.mlr.press/v139/radford21a.html>
- Türk Telekom. (2026a). 2025 annual report. Türk Telekom Investor Relations. Retrieved September 25, 2026, from <https://www.ttyatirimciiliskileri.com.tr/media/dqsjlvxo/2025-annual-report.pdf>
- Türk Telekom. (2026b). Hakkımızda [About us; operating figures at June 30, 2026]. Retrieved September 25, 2026, from <https://www.turktelekom.com.tr/hakkimizda/detay?p=tab3>
- Wang, S.-Y., Wang, O., Zhang, R., Owens, A., & Efros, A. A. (2020). CNN-generated images are surprisingly easy to spot... for now. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 8695–8704. Retrieved September 25, 2026, from <https://arxiv.org/abs/1912.11035>

9. Appendices

9.1 Development criteria

Table 5 lists the ten inherited numerical criteria, applied separately to original and social-style views. They are project-defined requirements, not an external certification standard. Automatic coverage and uncertainty are complementary, and several other metrics are correlated. The separate rule against newly missing any reference-detected AI image failed for E92. Coverage and uncertainty here describe each view separately; they are not the paired website outcomes in Section 4.5.6.

Table 5. E92 numerical development checks and their measured values.

Criterion	Requirement	Original	Social
Valid-score coverage	100%	100%	100%
ROC-AUC	≥ 0.90	0.9998	0.9942
Balanced accuracy	$\geq 85\%$	99.69%	95.31%
Pooled real false alerts	$\leq 10\%$	0.00%	8.75%
Worst real source	$\leq 20\%$	0.00%	18.75%
Pooled AI detection	$\geq 80\%$	99.38%	99.38%
Worst AI source	$\geq 60\%$	98.75%	98.75%
Automatic coverage	$\geq 80\%$	98.75%	98.44%
Accuracy when automatic	$\geq 95\%$	99.68%	95.24%
Uncertain fraction	$\leq 20\%$	1.25%	1.56%

9.2 Evidence and reproduction map

Full experiment settings, corrections and aggregate evidence remain in the repository snapshot cited as Keleş (2026). Private images and large weights are excluded from Git.